首届 openJiuwen 智能体创新大赛

openJiuwen

技术落地可行性方案

作品名称	AcceptAnything
作品定位	工程项目全链路测试验收工具
参赛方向	Coding 方向
文档编号	AA- TEC- 01
文档版本	2.0
编制日期	2026 年 11 月 3 日

编制说明

本方案论证在单机、自托管条件下实施工程项目验收的可行性。论证范围包括组件部署、业务用例执行、结果判定与故障恢复。评估依据为平台实现、三份冻结运行记录和一项本机索引基准。

目　录

第一章　部署条件与系统构成.....3

1.1　适用范围.....3

1.2　部署结构.....3

1.3　环境准备.....3

第二章　执行链路与判定设计.....4

2.1　需求提取与计划冻结.....4

2.2　运行隔离与局部失败.....4

2.3　结果契约与门禁.....4

第三章　验证结果与适用边界.....5

3.1　实仓验证.....5

3.2　覆盖核验与性能口径.....5

3.3　可行性结论.....5

第四章　试点实施与验收.....6

4.1　实施顺序.....6

4.2　故障恢复条件.....6

4.3　实现与记录索引.....6

第一章　部署条件与系统构成

1.1　适用范围

试点对象为能够取得指定版本源码、具备独立需求资料、允许在隔离环境中运行的工程。平台接受本地路径或 GitHub、GitLab 仓库地址。远程分支在验收前解析到确定的提交。运行记录还保存源码快照摘要，以便确认后续复验是否使用同一组文件。

1.2　部署结构

图 1.1 按进程职责划分控制、执行和读取三部分。工作台提交验收范围，API 建立任务记录，工作进程准备环境并调用模型和测试工具。结果读取进程依据本轮归档生成验收视图。读取历史结果时无需重新请求模型，也不应重新执行目标仓库的代码。

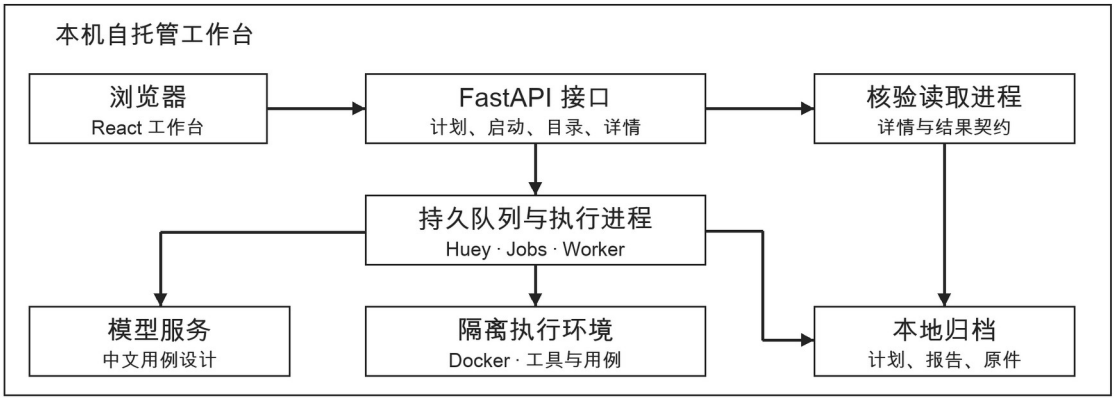


图 1.1　单机部署结构

1.3　环境准备

表 1.1　单机试点的运行依赖

依赖项目	配置及检查方法
平台运行时	Python 3.12、Node.js 24 或以上。按锁文件安装后端和前端依赖，启动本机工作台。
隔离执行	准备 Docker 引擎、目标技术栈镜像和 Chromium。用容器中的测试服务验证启动、连接和清理。
模型服务	配置可用的模型接口。登记请求标识与结果；提供方未返回终态时保留未知状态。
归档存储	为快照、日志、报告、HAR 和浏览器回放分配磁盘。容量由目标工程和留存周期决定。

第二章　执行链路与判定设计

2.1　需求提取与计划冻结

需求文档给出待验证的行为。平台使用仓库清单、公开路由和代码签名定位调用入口，模型根据带出处的需求编写中文用例。计划确认前，系统校验原文引用、用例编号、前置条件和动作协议。例如，接口签名可以确定参数名称，但无法单独确定无效输入应返回哪个状态码。

2.2　运行隔离与局部失败

服务编排按依赖顺序启动并探测就绪。每条业务用例从准备基线取得独立工作环境，防止前一用例写入的数据库影响下一条断言。准备失败时，系统将原因记在所依赖的服务或组件上，将可独立运行的组件继续调度。图 2.1 给出了环境分配和回收边界。

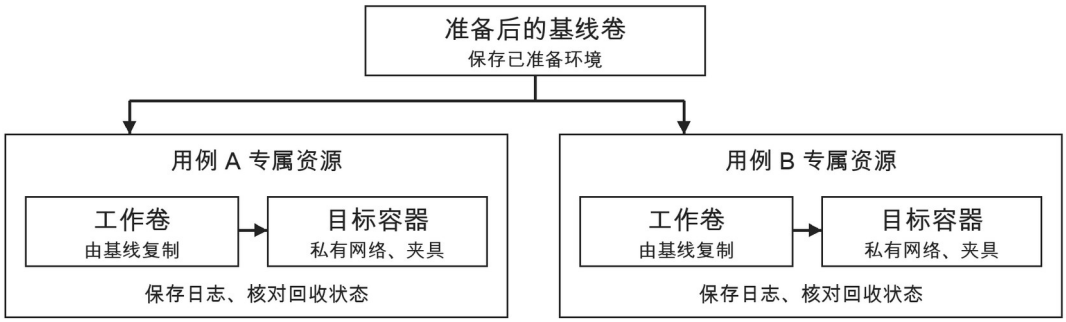


图 2.1　用例环境的分配及回收

2.3　结果契约与门禁

acceptance/results.py 中的 gate_blockers 函数核对任务与规则标识、冻结计划的任务集合、报告完整性，以及必验用例与有效观察的对应关系。project 函数同时返回运行状态、覆盖情况、发现分类和门禁。原件校验失败、必验项没有观察或任务未结束，均会使当前门禁为 inconclusive；已记录决定仍保留。

第三章　验证结果与适用边界

3.1　实仓验证

表 3.1 统计模型业务用例，未计入仓库原生测试。这三轮使用已封存的模型响应重新准备环境并执行。Fastify 采用追加复跑 int-9ebeca89d0194c8d，原先因下载失败而中断的记录单独保留。

表 3.1　三个仓库的业务执行结果

仓库	通过	失败	异常	当前门禁
Flask	68	3	14	无法判定
Fastify	16	1	2	无法判定
sqlite-utils	64	6	2	无法判定

3.2　覆盖核验与性能口径

以 Fastify 为例，该轮 19 项任务均已结束，结果契约的 37 条需求中，14 条已验证、9 条已排除、14 条未验证。此处的“已验证”表示必需的观察已取得并通过完整性检查，断言本身仍可能失败。任务数、需求数和用例数分别计数。

单机目录基准使用 M5 Pro、48 GiB 内存与本地 SQLite。10,000 条合成记录冷建索引耗时 23.297 秒；首页读取 50 条，30 次热查询 P95 为 120.38 毫秒。该实验只测量历史目录查询，浏览器并发容量和单仓库费用须在试点机另行测量。

3.3　可行性结论

现有记录支持将系统用于受控的单机工程验收试点。Redis、Celery 和 Web 的跨服务路径已有同一任务标识的请求及 worker 日志可核。引入 PostgreSQL 或 MongoDB 前，需要分别验证凭据发放、迁移、种子数据、就绪探测和清理。共用服务器上的权限隔离、脱敏和长期留存仍是上线前条件。

第四章　试点实施与验收

4.1　实施顺序

部署人员在干净机器上安装并记录版本。测试负责人先执行含已知结果的正例和反例，再选择一个具备独立需求的工程。项目负责人签收本轮源码、依据和排除项。试点结束后交付原始报告、用例明细、覆盖账本、资源清理记录及门禁结论。

表 4.1　试点验收与故障检查

检查项	操作	验收条件
错误预期	选择可控的反例输入	观察与预期不符时断言失败，原因能回查
证据失效	在测试副本移除必验报告	重新核验给出无法判定，旧决定可查
进程中断	中断已领取任务后恢复服务	区分待投递与已开始的尝试，保留清理状态
费用与容量	分别记录模型调用、准备和执行资源	以目标机实测建立试点预算及留存周期

4.2　故障恢复条件

任务领取、心跳和尝试标识保存于本机。恢复时核查原进程与归属资源；已开始且无法确认终态的尝试不得自动重放。若 Docker 暂不可用而无法确认清理结果，保留恢复错误，待处理后再签收。对于已发送但上游未返回结果的模型调用，还应独立核对供应方记录。

4.3　实现与记录索引

[1] README.md，部署入口与依赖说明。 [2] acceptance/universal/orchestration.py、case_environment.py，服务图和环境分配。 [3] acceptance/results.py、jobs.py，结果投影与任务恢复。

[4] docs/design/d041/r6-交付结果与剩余边界.md，三仓库实测。 [5] docs/design/d043/ 目录内三份运行结果契约及 history-benchmark.json，覆盖统计和基准口径。